| | |
|---|---|
| $& → String | The string matched (following a successful pattern match). This variable is local to the current scope. [r/o, thread] |
| $+ → String | The contents of the highest-numbered group matched following a successful pattern match. Thus, in "cat" =~ /(c\|a)(t\|z)/, $+ will be set to "t." This variable is local to the current scope. [r/o, thread] |
| $` → String | The string preceding the match in a successful pattern match. This variable is local to the current scope. [r/o, thread] |
| $' → String | The string following the match in a successful pattern match. This variable is local to the current scope. [r/o, thread] |
| $1...$n → String | The contents of successive groups matched in a pattern match. In "cat" =~ /(c\|a)(t\|z)/, $1 will be set to "a" and $2 to "t." This variable is local to the current scope. [r/o, thread] |
| $~ → MatchData | An object that encapsulates the results of a successful pattern match. The variables $&, $`, $', and $1 to $9 are all derived from $~. Assigning to $~ changes the values of these derived variables. This variable is local to the current scope. [thread] |

## Character Classes

If you look at the table, you'll see that some of the character classes have different interpretations depending on the character set option defined for the regular expression. These options tell the regexp engine whether (for example) word characters are just the ASCII alphanumerics, or whether they should be extended to include Unicode letters, marks, numbers, and connection punctuation. The options are set using the sequence (?_option_), where the option is one of d , a for ASCII-only support, and u for full Unicode support. If you don't specify an option, it defaults to (?d).

```
show_regexp('über.', /(?a)\w+/) # => ü->ber<-.
show_regexp('über.', /(?d)\w+/) # => ü->ber<-.
show_regexp('über.', /(?u)\w+/) # => ->über<-.
show_regexp('über.', /(?d)\W+/) # => ->ü<-ber.
show_regexp('über.', /(?u)\W+/) # => über->.<-
```

The POSIX character classes, as shown in Table 10, POSIX character classes, on page 445, correspond to the ctype(3) macros of the same names. They can also be negated by putting an up arrow (or caret) after the first colon:

```
show_regexp('Price $12.', /[aeiou]/)          # => Pr->i<-ce $12.
show_regexp('Price $12.', /[[:digit:]]/)      # => Price $->1<-2.
show_regexp('Price $12.', /[[:space:]]/)      # => Price-> <-$12.
show_regexp('Price $12.', /[[:^alpha:]]/)     # => Price-> <-$12.
show_regexp('Price $12.', /[[:punct:]aeiou]/) # => Pr->i<-ce $12.
```

These versions are much rarer in actual Ruby code, in our experience. However, the POSIX classes do match non-ASCII characters.

You can create the intersection of character classes using &&. So, to match all lowercase ASCII letters that aren't vowels, you could use this:

```
str = "now is the time"
str.gsub(/[a-z&&[^aeiou]]/, '*') # => "*o* i* **e *i*e"
```